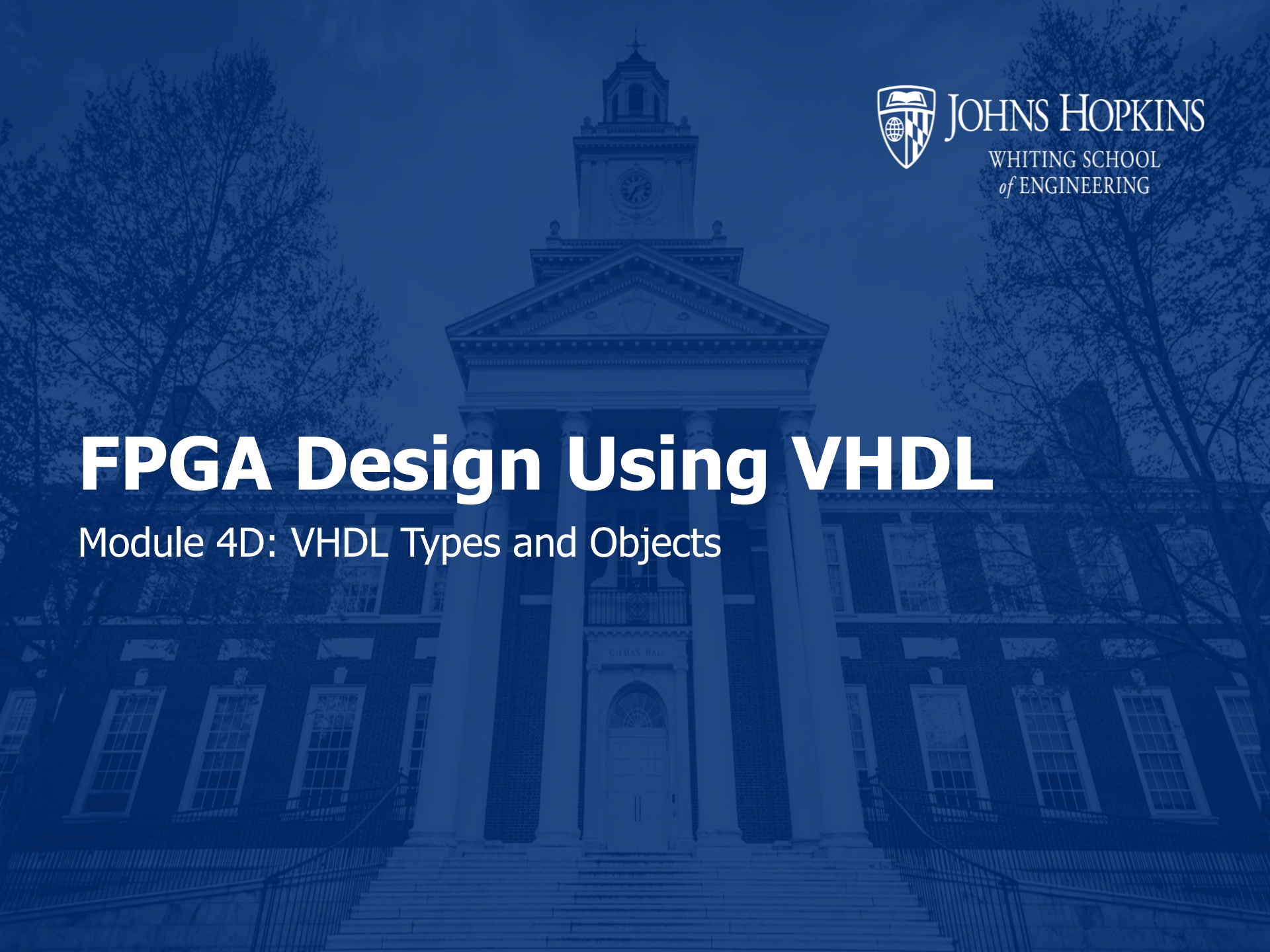

FPGA Design Using VHDL

Module 4D: VHDL Types and Objects

Types vs. Objects

- To clarify vocabulary, an **object** is a named item in a VHDL model that has a value of a specified **type**
- There are five (5) “classes” of objects:

1. Constants (i.e. constant PERIOD : time := 100 ns;)
2. Variables (i.e. variable cntInt : integer;)
3. Signals (i.e. signal notAClk : unsigned;)
4. Ports (i.e. inPort : in std_logic;)
5. Files (i.e. file notCoveringTheseToday : text;)

Object **Type**

VHDL Datatypes

- VHDL is a strongly typed language
  - Strongly typed means “every object may only assume values of its nominated type”
  - Allows detection of errors at an early stage of design
  - Each object has a type associated with it,
    - No ambiguity
  - Converting types may be necessary, even those that are very similar

Operators (Briefly)

- Operations can be performed on signals, variables, and other objects
- The operators available fall into five (5) categories:
  1. Boolean: AND, OR, NAND, NOR, XOR, XNOR
  2. Comparison: =, /= (not equal), <, <=, >, >=
  3. Shifting: sll, srl, rol, ror (available post VHDL ‘87) VHDL'08 supports std_logic_vector
  4. Arithmetic: sign +, sign -, abs, +, -, *, /, mod, rem, ** (power)
  5. Miscellaneous: Concatenation (&), NOT

Bit Type

- Type “bit” is a built-in logical type:
  - Type `bit` is (‘0’, ‘1’);
  - What’s the difference between `bit` and `std_logic`?
- Both the Boolean and Comparison operators apply to bit
  - `sig_bit <= sig1_bit (and/or/nor) sig2_bit;`
  - `if (sig1_bit = sig2_bit) …`
  - `if (sig1_bit = sig2_stdlogic) … -- not allowed`

Boolean Type

- Boolean is the comparison type in VHDL
  - All comparison operators result in a Boolean value,
    - `if (sig1_bit = sig2_bit) … -- returns a Boolean value`
  - It is not possible to use any other type in a conditional test
    - Needs to evaluate to `TRUE` or `FALSE`
    - Not allowed: `if (a)` where `a` is of type bit or `std_logic`, as an example

      Exception: VHDL '08 supports implicit conversion of std_logic to boolean

Boolean Type Continued

- Boolean is synthesizable,
- Boolean useful in designs and testbenches where it represents what it is – either true or false
  - Typically used as constants
    - Can use as signals and variables:
    - `constant use_mem : boolean := true;`
      - `if(use_mem)  -- this is allowed`
    - `signal bool_s : boolean := true;`
    - `variable bool_v : boolean := false;`

Signed/ Unsigned Types

- `signed` and `unsigned` are arrays of `std_logic`, defined in the `numeric_std` library
  - `unsigned` range [0, 2^N-1]
  - `signed` range [-2^{N-1}, 2^{N-1}-1]

Signed/ Unsigned Types cnt'd

- Cannot mix types in an operation or statement
  - `A_signed32 <= B_unsigned32; -- illegal`
  - `A_signed32 <= B_signed32 + 1; -- fine`
- You need to convert between `signed/unsigned` and `std_logic_vector`
  - `A_slv16 <= std_logic_vector(B_unsigned16);`
  - You may need to do something like that for Lab 4

Integer Types

- `integer` is built-in numeric type which represents integer values
  - **Most implementations presume a 2's complement representation:**
    - **The range is the [–2147583648, 2147583647]**
  - **Good practice is to force a range on integer types**
    - `signal cntr : integer range 0 to 15;`
- Comparison and arithmetic operators are available for modeling
- **Integer types are often (generally) used as an index to a vector or array**

Enumerated Types

- User specifies list of possible values
- May be specified to either signals or variables
- In this class, very useful for Finite State Machines (FSM)
- An enumerated type is a type composed of a set of literal values or character literals
  - `type terLogic is ('H', 'L', 'ZL', 'ZH');`
  - `signal state_x : terLogic ;`

Arrays

- An array is a collection of elements, all of which are the same type
- For example:
  - A constrained array definition:

```
type AA is array(6 downto 0) of std_logic_vector(5 downto 0);
signal r : AA; -- r(2) for entire std_logic_vector or
               -- r(0)(0) to r(6)(5) bit-wise
```

  - Multidimensional array definition:

```
type ARR is array (1 downto 0, 1 downto 0) of std_logic_vector(3
downto 0);
signal my_array : ARR;
-- access my_array (0,0) <= "0000";
-- access my_array (1,1) <= "0000";
```

my_array

my_array(0,0) → std_logic_vector(3 downto 0)	my_array(0,1) → std_logic_vector(3 downto 0)
my_array(1,0) → std_logic_vector(3 downto 0)	my_array(1,1) → std_logic_vector(3 downto 0)

Std_logic_vector Assignments

```
-- only defined signals that knowing there
-- definition is neccesary for the example
signal up   : std_logic_vector(1 to 4);
signal down : std_logic_vector(4 downto 1);
Signal a    : std_logic_vector(0 to 3);
Signal b    : std_logic_vector(3 downto 0);
Signal both : std_logic_vector(7 downto 0);
Signal long : std_logic_vector(63 downto 0);
signal item : natural range 0 to 3;

begin

   -- this...
   down <= up;
   -- ...is equivalent to this
   down(1) <= up(4);
   down(2) <= up(3);
   down(3) <= up(2);
   down(4) <= up(1);

   -- we can index with the indices
   -- which is known as static indexing
   a(0) <= '1';

   -- we can use the operators defined for
   -- the individual element type of the array
   sum(0)<= a(0) xor b(0) xor cin;

   -- we can dynamically index into an array
   -- the array is accessed by the value of item
   a(item) <= '0';
```

```
   -- we can "slice" an array
   -- remember the elements are assigned
   -- left to right, regardless of the
   -- indices, so this...
   b(1 downto 0) <= a(2 to 3);
   -- ...is equivalent to this
   b(1) <= a(2);
   b(0) <= a(3);
   .
   .
   -- aggragates
   -- copy "1000" to b
   b <= (3 =>'1',2 =>'0',1 =>'0',0 => '0');

   -- another copy (copy "0001" to b)
   b <= (0 => '1', 1 =>'0',2 =>'0',3 =>'0');

   -- other useful examples
   b <= ('1', '0', '0', '0');
   a <= (3 => '1', others => '0');

   -- concatenate the up and down arrays
   both <= up & down;

   -- assign a std_logic_vector with hex values
   long <= X"12ABDF2311562312";

   -- assign with hex values and binary values
   both <= X"A" & "0101";
   .
   .
   .
```

Std_logic_vector Assignments Continued

```
-- only defined signals that knowing there
-- definition is neccesary for the example
signal up   : std_logic_vector(1 to 4);
signal down : std_logic_vector(4 downto 1);
Signal a    : std_logic_vector(0 to 3);
Signal b    : std_logic_vector(3 downto 0);
Signal both : std_logic_vector(7 downto 0);
Signal long : std_logic_vector(63 downto 0);
signal item : natural range 0 to 3;

begin

   -- this...
   down <= up;
   -- ...is equivalent to this
   down(1) <= up(4);
   down(2) <= up(3);
   down(3) <= up(2);
   down(4) <= up(1);

   -- we can index with the indices
   -- which is known as static indexing
   a(0) <= '1';

   -- we can use the operators defined for
   -- the individual element type of the array
   sum(0)<= a(0) xor b(0) xor cin;

   -- we can dynamically index into an array
   -- the array is accessed by the value of item
   a(item) <= '0';
```

```
   -- we can "slice" an array
   -- remember the elements are assigned
   -- left to right, regardless of the
   -- indices, so this...
   b(1 downto 0) <= a(2 to 3);
   -- ...is equivalent to this
   b(1) <= a(2);
   b(0) <= a(3);
   .
   .
   -- aggragates
   -- copy "1000" to b
   b <= (3 =>'1',2 =>'0',1 =>'0',0 => '0');

   -- another copy (copy "0001" to b)
   b <= (0 => '1', 1 =>'0',2 =>'0',3 =>'0');

   -- other useful examples
   b <= ('1', '0', '0', '0');
   a <= (3 => '1', others => '0');

   -- concatenate the up and down arrays
   both <= up & down;

   -- assign a std_logic_vector with hex values
   long <= X"12ABDF2311562312";

   -- assign with hex values and binary values
   both <= X"A" & "0101";
   .
   .
   .
```